CodeGuardian: Policy-Aware Secure Coding Assistant with LLM and Tool Invocation

Suthar Vishalkumar Jadeshbhai 79002

Syed Abdullah 78744

Verda Janset Guvel 78709

Rana Kavak 78710

Serra Nur Yucel 78715

Claude Cyubahiro 77791

Abstract

Modern software development workflows frequently introduce security vulnerabilities due to tight deadlines, insufficient security expertise, and the fragmented nature of existing static analysis tools. Traditional scanners such as Bandit and Semgrep detect common weaknesses but often generate high false-positive rates, lack contextual reasoning, and fail to enforce organization-specific security policies. Recent research in policy-aware large language models (LLMs) demonstrates strong potential for adapting code generation and review to predefined security standards, while Model Context Protocol (MCP) enables LLMs to safely invoke external tools for grounded, verifiable analysis.

This project presents CodeGuardian, an IDE-integrated secure coding assistant that combines LLM-based reasoning, real-time static analysis, and policy interpretation to proactively detect and remediate software vulnerabilities during development. CodeGuardian incorporates a hybrid architecture: (1) an LLM Orchestrator capable of understanding code context and triggering external vulnerability scanners through MCP; (2) a Policy Engine that aligns code recommendations with OWASP Top 10, CERT guidelines, and custom organizational rules; and (3) a Remediation Engine that generates policy-compliant fixes with clear explanations. The system is embedded into VSCode/JetBrains, providing real-time alerts, secure alternatives, and developer-friendly feedback.

A companion dashboard summarizes vulnerabilities, policy violations, and remediation history, enabling teams to track security posture across projects. Evaluation will measure CodeGuardian's effectiveness in reducing vulnerability density, lowering false-positive rates, and improving developer trust and understanding compared to traditional static analysis workflows. By integrating LLM reasoning with verifiable tool outputs and explicit security policies, CodeGuardian advances the state of secure-by-design software engineering.

1 Introduction

Modern software development relies heavily on fast-paced iteration cycles, large codebases, and increasingly complex software ecosystems. As applications grow in scale and interconnectivity, the risk of introducing security vulnerabilities during development increases substantially. Developers frequently write insecure code due to time pressure, limited security training, or reliance on outdated practices. Common vulnerabilities—such as injection flaws, insecure deserialization, improper authentication, and unsafe cryptographic usage—can compromise system integrity, user privacy, and organizational security. Ensuring robust, secure-by-design software therefore requires tools that provide immediate, contextual, and policy-aligned guidance during the coding process.

1.1 Motivation and Problem Statement

Despite the availability of static analysis tools and linters, developers continue to struggle with secure coding due to several persistent challenges:

- High false-positive rates in existing static analyzers (e.g., Semgrep, Bandit), leading to alert fatigue
- Lack of contextual reasoning, causing tools to miss vulnerabilities dependent on global or multi-file logic
- Difficulty interpreting and applying security policies (e.g., OWASP Top 10, CERT guidelines)
- Limited integration into real-time development workflows, causing late discovery of vulnerabilities
- Inconsistent or incomplete remediation suggestions, requiring developers to manually research secure alternatives

These limitations create a critical need for a secure coding assistant that can analyze code in real time, understand developer intent, interpret organizational security rules, and provide actionable and trustworthy remediation guidance.

1.2 Technological Context

Recent advances in large language models (LLMs) and policy-aware AI systems have introduced new opportunities for enhancing software security during development. Models such as GPT-4.1 and research in policy-aware code generation demonstrate strong potential for:

- Understanding program structure and intent
- Detecting vulnerabilities beyond simple rule-based pattern matching
- Interpreting formal security standards and organizational policies
- Generating contextual, high-quality secure code recommendations

Simultaneously, emerging protocols such as the Model Context Protocol (MCP) enable LLMs to safely invoke external tools—including static analyzers, dependency checkers, and policy evaluators—creating hybrid systems that combine AI reasoning with verifiable tool-driven outputs.

However, current solutions remain fragmented:

- LLMs alone may hallucinate or overlook critical vulnerabilities
- Static analyzers lack contextual understanding
- Existing IDE plugins fail to combine policy reasoning, tool invocation, and remediation into a unified workflow

This gap motivates the development of an integrated, policy-aware, tool-augmented secure coding assistant.

1.3 Proposed Solution

We propose **CodeGuardian**, a real-time, IDE-integrated secure coding assistant that combines LLM-based reasoning with static analysis and explicit policy enforcement. CodeGuardian enables developers to write secure code by providing:

- Real-time vulnerability detection through hybrid LLM + scanner analysis
- Policy-aware reasoning aligned with OWASP Top 10, CERT guidelines, and organizational rules

- Actionable code remediation, offering corrected code snippets with detailed explanations
- Safe tool invocation using MCP to run Bandit, Semgrep, and other analyzers
- Developer-friendly IDE integration with inline warnings and interactive suggestions
- Team-wide security visibility through a dashboard summarizing vulnerability trends and remediation history

By unifying policy interpretation, vulnerability scanning, and contextual LLM reasoning, CodeGuardian promotes secure-by-design practices and reduces reliance on late-stage security reviews.

1.4 Contributions

This paper contributes:

1. A systematic literature review synthesizing 15–20 research papers on policy-aware LLMs, static analysis tools, and secure coding assistance
2. The **CodeGuardian** system architecture integrating LLM reasoning, policy engines, MCP-based tool invocation, and IDE workflows
3. A detailed implementation plan including architecture diagrams, workflow charts, and a data model
4. An evaluation methodology comparing CodeGuardian's hybrid approach against traditional static analysis tools in terms of vulnerability detection accuracy, false-positive reduction, and developer usability

Together, these contributions provide a comprehensive framework for an intelligent, policy-compliant, developer-centered secure coding assistant.

2 Literature Review

This section presents a structured review of 18 research papers related to secure code generation, vulnerability detection, LLM-assisted analysis, and policy-aware software development. These works were collected from sources such as arXiv, IEEE Xplore, SpringerLink, and ResearchGate, using targeted keywords including "secure coding", "vulnerability detection", "large language models", "static analysis", and "policy-aware AI". The following subsections summarize the main contributions, methodologies, and limitations found in the existing body of work.

2.1 Policy-Aware Large Language Models

2.1.1 A Comprehensive Study of LLM Secure Code Generation (2025)

What the authors did: Evaluated how well LLMs generate secure code across multiple languages and security categories.
How they did it: Used benchmark datasets with known vulnerabilities and measured vulnerability density in generated code.
Pros: Multi-language evaluation; identifies common insecure patterns; strong empirical analysis.
Cons: No integration with static analysis tools; limited policy enforcement.

2.1.2 Toward Secure Code Generation with LLMs (Apiiro, 2025)

What the authors did: Investigated the role of contextual prompts and policies in secure LLM code generation.
How they did it: Analyzed code produced under various security-aware prompting strategies.
Pros: Demonstrates the importance of explicit security context; practical recommendations.
Cons: Not evaluated in real IDE workflows; limited tool-based validation.

2.1.3 Do LLMs Consider Security? (Springer, 2025)

What the authors did: Explored whether LLMs naturally follow secure coding practices.
How they did it: Conducted controlled experiments with insecure and ambiguous coding prompts.
Pros: Strong evidence-based analysis; highlights gaps in security awareness.
Cons: Focuses only on code generation, not vulnerability detection.

2.2 LLM-Assisted Vulnerability Detection

2.2.1 LLMLOOP: Improving LLM-Generated Code and Tests (ICSME 2025)

What the authors did: Proposed a framework to refine LLM-generated code using testing plus static analysis feedback loops.
How they did it: Applied automatic test generation and static analysis, then fed results back into the LLM.
Pros: Reduces insecure patterns; hybrid approach improves reliability.
Cons: Feedback loops are slow; not suited for real-time IDE usage.

2.2.2 Security and Quality in LLM-Generated Code (2025)

What the authors did: Evaluated LLMs for security, correctness, and maintainability across languages.
How they did it: Used static analysis metrics and human review.
Pros: Comprehensive multi-language dataset.
Cons: No integration with policy engines.

2.2.3 Code Vulnerability Detection: Comparative LLM Analysis (2024)

What the authors did: Compared GPT, Llama, and other LLMs for vulnerability detection.
How they did it: Used labeled datasets to evaluate detection accuracy across vulnerability types.
Pros: Identifies strengths of LLM-based reasoning.
Cons: High hallucination rate; lacks tool grounding.

2.3 Static Analysis and ML-Based Detection

2.3.1 Static Analysis of Source Code Vulnerability Using ML Techniques (2022)

What the authors did: Provided a systematic review of ML-based vulnerability detection.
How they did it: Compared supervised, unsupervised, and hybrid detection methods.
Pros: Helpful taxonomy of ML techniques for vulnerability detection.
Cons: No LLM integration; ML models struggle with code semantics.

2.3.2 LEOPARD: Identifying Vulnerable Code (2019)

What the authors did: Presented a program-metrics-based approach to locate vulnerable functions.
How they did it: Extracted static code metrics and applied ranking algorithms.
Pros: Lightweight and complementary to static analysis.
Cons: Limited semantic understanding; relies on heuristics.

2.3.3 VELVET: Ensemble Learning for Vulnerability Detection (2021)

What the authors did: Proposed an ensemble learning approach for detecting vulnerable statements in source code.
How they did it: Combined multiple ML models with program analysis techniques.
Pros: Improves precision and recall; advanced ML integration.
Cons: High computational cost; limited real-time use.

2.4 Organizational Security Policies

2.4.1 OWASP Top 10 (2021)

What the authors did: Defined the most critical web application security risks.
How they did it: Consolidated data from industry trends and real vulnerability reports.
Pros: Universally adopted; essential baseline for secure coding.
Cons: High-level; requires mapping to code-specific rules.

2.4.2 CERT Secure Coding Guidelines (SEI, ongoing)

What the authors did: Provided language-specific secure coding rules for C, C++, Java, and Python.
How they did it: Derived rules from empirical vulnerabilities and language semantics.
Pros: Very practical and precise; widely respected.
Cons: Too detailed for novice developers without automation.

2.4.3 Secure Coding Best Practices (NIST, 2022)

What the authors did: Published guidelines for secure development lifecycle (SDLC).
How they did it: Synthesized practices from federal agencies and software vendors.
Pros: Strong alignment with enterprise requirements.
Cons: No IDE-level guidance; process-oriented rather than code-oriented.

2.5 Explainable AI for Software Security

2.5.1 Explainable Static Analysis (2022)

What the authors did: Studied how explanations affect developer trust in security tools.
How they did it: Tested multiple explanation styles on developers.
Pros: Clear evidence that explanations improve adoption.
Cons: Focused on classical tools, not LLMs.

2.5.2 Human-AI Collaborative Debugging (Microsoft, 2023)

What the authors did: Investigated how developers interact with AI debugging systems.
How they did it: Conducted user studies on multi-step debugging assistance.
Pros: Shows developers prefer step-by-step reasoning.
Cons: Not security-specific; lacks policy integration.

2.5.3 AI for Code Synthesis: Can LLMs Generate Secure Code? (2024)

What the authors did: Explored whether LLMs can generate secure-by-default code.
How they did it: Evaluated LLM code using automated vulnerability scanning.
Pros: Useful for understanding LLM weaknesses.
Cons: No policy-driven remediation; limited tool integration.

3 Gap Analysis

The findings from our review of 18 research papers highlight several unresolved challenges in secure code generation, LLM-based vulnerability detection, and policy-aware development workflows. These gaps directly shape the design objectives of the proposed CodeGuardian system.

1. **Lack of Consistent Policy Enforcement in LLM Code Generation:** Existing studies show that large language models often produce syntactically correct code but fail to consistently follow established security guidelines such as OWASP or CERT. Most LLMs rely heavily on prompt wording, resulting in unpredictable adherence to security standards.

2. **Limited Integration of LLMs with Static Analysis Tools:** While LLMs can reason about vulnerabilities, they frequently hallucinate or overlook subtle issues. At the same time, static analysis tools provide accurate but context-blind warnings. Few systems combine both strengths in a unified, real-time environment.

3. **Insufficient Support for Real-Time, IDE-Embedded Security Feedback:** Much of the existing research evaluates LLMs offline using benchmark datasets. Very few solutions integrate secure code generation directly into an IDE workflow, where developers actually write and test code. This leaves a gap in providing immediate, actionable feedback during coding.

4. **Weak Explanatory Support for Developers:** Many security tools highlight vulnerabilities but provide minimal explanation or reasoning. Research shows that developers benefit from clear, actionable guidance. Current systems lack transparent, LLM-powered explanations that connect detected issues with relevant security policies.

These gaps demonstrate the need for an integrated solution like CodeGuardian, which combines policy-aware reasoning, static analysis, real-time feedback, and developer-friendly explanations within a unified IDE assistant.

4 Proposed System: CodeGuardian

CodeGuardian is designed as an intelligent, security-focused assistant that supports developers during the software development process by identifying vulnerabilities early, enforcing security policies, and providing clear remediation suggestions. The system combines language-model reasoning, automated code analysis tools, and organization-defined security standards to create a unified environment where secure development becomes the default rather than an afterthought. CodeGuardian operates directly inside the developer's IDE and offers continuous, context-aware guidance without disrupting the natural coding workflow.

4.1 System Architecture

The overall architecture of CodeGuardian consists of multiple interconnected layers that work together to deliver real-time, policy-driven security feedback. At a high level, the system includes an IDE extension responsible for capturing code changes, a backend engine that orchestrates

vulnerability detection tasks, a policy engine that interprets organizational rules, and an LLM module that generates explanations and secure code alternatives. These components exchange information through a controlled function-invocation mechanism, ensuring both accuracy and security. The architecture diagram in Figure **??** provides an overview of how data flows through the system.

5 System Architecture

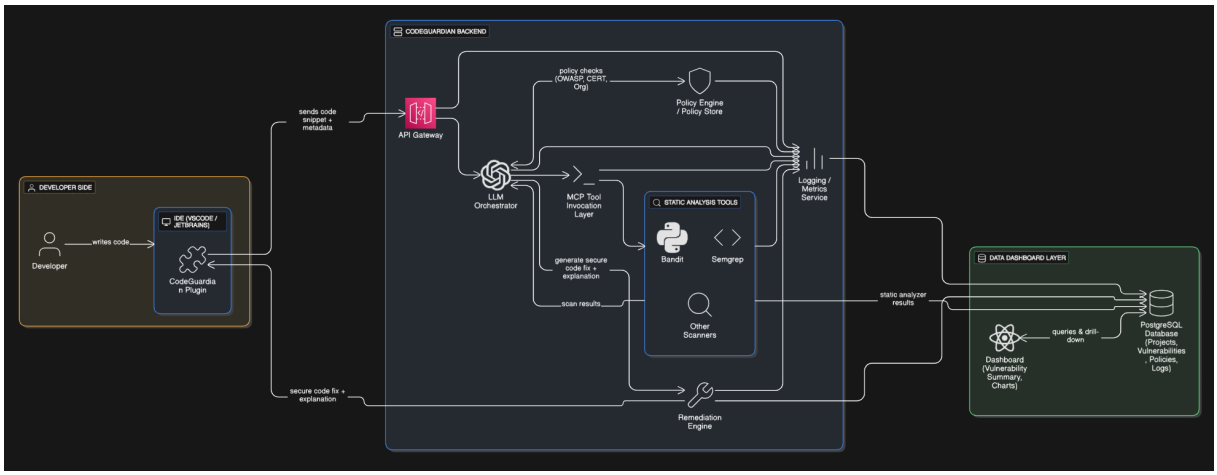

Figure 1: System architecture of CodeGuardian integrated into the developer's IDE.

5.1 Architecture Description

The CodeGuardian architecture connects the developer's IDE with a secure, policy-aware backend. When the developer writes code, the IDE plugin sends the relevant snippet and metadata to the backend through the API Gateway. The LLM Orchestrator receives this input and coordinates between two main components: the Static Analysis Tools and the Policy Engine. Tools such as Bandit and Semgrep generate concrete vulnerability reports, while the Policy Engine checks the code against OWASP, CERT, and organization-specific rules. The LLM combines these outputs to produce clear explanations and secure code fixes. All results are logged for analysis and also stored in a PostgreSQL database, which powers the dashboard where developers and teams can review vulnerabilities, trends, and project history.

5.2 System Workflow

The CodeGuardian workflow describes how a code change moves from the developer's IDE through the analysis pipeline and back as actionable security feedback.

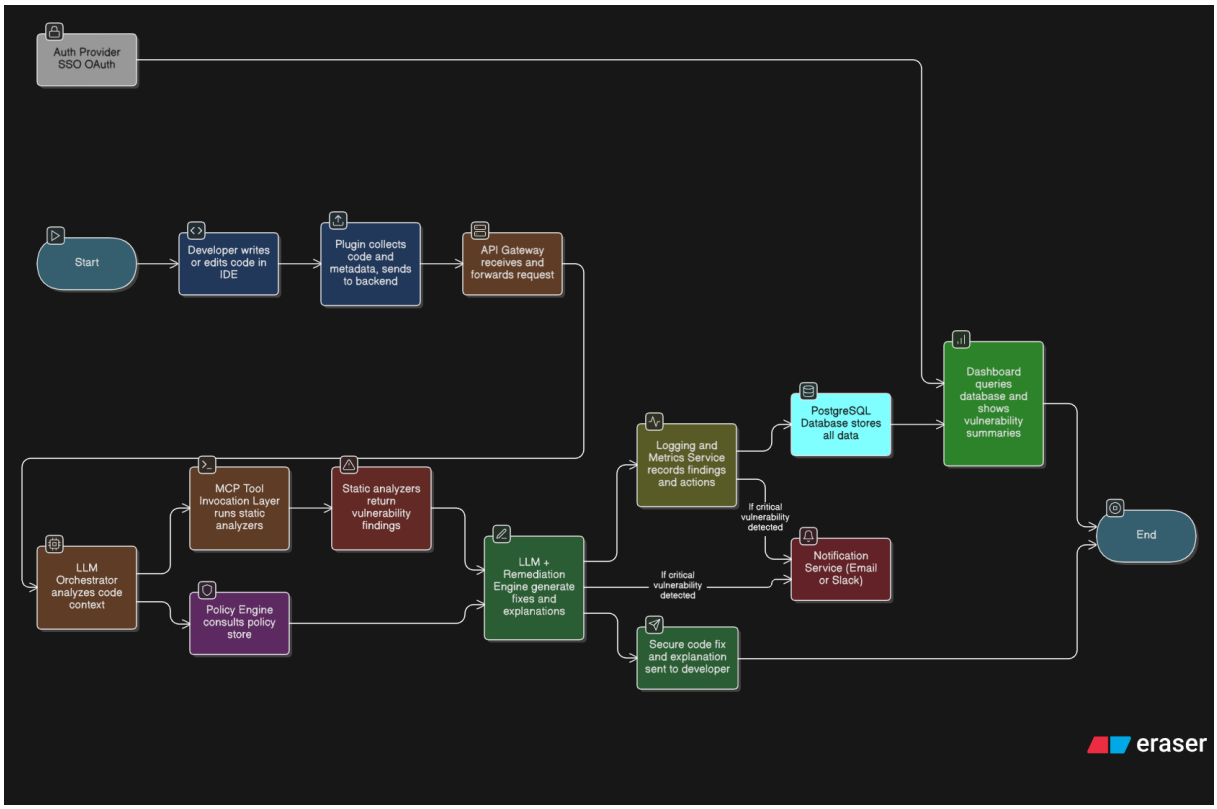

Figure 2: Workflow of the CodeGuardian secure coding pipeline.

1. **Code editing in the IDE:** The process starts when the developer writes or edits code in VSCode or JetBrains with the CodeGuardian plugin installed.

2. **Plugin request to backend:** On save or on demand, the plugin collects the relevant code snippet and metadata, then sends a request to the backend through the API Gateway.

3. **Request routing and orchestration:** The API Gateway receives the request and forwards it to the LLM Orchestrator, which interprets the context of the code and decides which analysis actions are required.

4. **Static analysis invocation:** Via the MCP Tool Invocation Layer, the orchestrator runs static analyzers such as Bandit, Semgrep, and other scanners on the submitted code.

5. **Policy consultation:** In parallel, the Policy Engine consults the policy store, applying OWASP, CERT, and organization-specific rules to both the code and the scanner findings.

6. **Findings aggregation and remediation:** The LLM and Remediation Engine combine static analysis results with policy checks to generate secure code fixes and human-readable explanations.

7. **Logging and storage:** The Logging and Metrics Service records all findings, actions, and severity levels. This data is stored in the PostgreSQL database together with project, policy, and historical vulnerability information.

8. **Dashboard and authentication:** Authorized users access the dashboard via an external authentication provider (e.g., SSO/OAuth). The dashboard queries the PostgreSQL database to display vulnerability summaries, trends, and drill-down views.

9. **Notifications for critical issues:** If a critical vulnerability is detected, a notification service (such as email or Slack) is triggered to alert security analysts or responsible stakeholders.

10. **Feedback to the developer:** Finally, the secure code fix and explanation are sent back through the API Gateway to the IDE plugin, which highlights the affected code and presents the recommended remediation to the developer, ending the workflow.

5.3 Entity–Relationship (ER) Diagram

The ER diagram provides an overview of how data is organized in the CodeGuardian system. Users can belong to multiple projects through the ProjectMember relationship, which stores each user's role in that project. Projects contain source files and scan records, and each scan produces one or more findings. A finding is linked to both the file where it was detected and the policy rule it violates. When an automatic fix is available, it is stored as a Remediation connected to that finding. Notifications are used to alert users about important findings. Together, these entities represent the core structure needed for tracking vulnerabilities, enforcing security policies, and managing developer interactions.

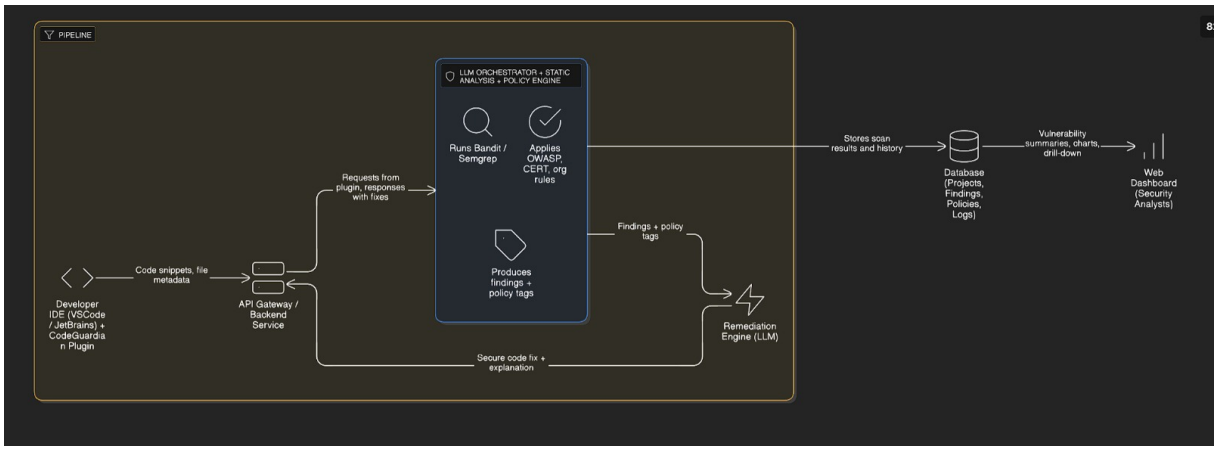

Figure 3: ER diagram of the CodeGuardian data mode.

5.4 Technology Stack

Table 1: Core technology stack and justification for CodeGuardian.

Component	Technology	Justification
IDE Integration	VSCode / JetBrains Plugin	Provides real-time feedback, inline vulnerability alerts, and seamless integration into developer workflows.
Backend API	FastAPI	Fast, asynchronous backend service that handles requests, routes analysis tasks, and supports scalable APIs.
LLM Orchestrator	GPT-4o / LLM Engine	Interprets code context, combines scanner results, and generates secure remediation suggestions.
Static Analysis Tools	Bandit, Semgrep	Covers Python and multi-language vulnerability scanning; provides deterministic rule-based detection.
Policy Engine	OWASP / CERT Rules	Applies standardized security policies to ensure consistent and compliant vulnerability evaluation.
Database	PostgreSQL	Stores users, projects, scan results, vulnerabilities, policies, and system logs reliably within a relational model.
Dashboard	React / Next.js	Enables security analysts and teams to visualize vulnerability trends, project history, and compliance status.

6 System Evaluation Plan

This section outlines how CodeGuardian will be evaluated in terms of vulnerability detection, policy enforcement, and remediation quality.

6.1 Evaluation Setup

To conduct a realistic assessment, we prepare a collection of test repositories that include:

- Open-source projects in Python, JavaScript, and Java
- Intentionally vulnerable sample code based on OWASP and CERT patterns
- Synthetic snippets designed to test edge cases and LLM reasoning
- A defined policy set including OWASP, CERT, and organization-specific rules

6.2 Evaluation Procedure

The evaluation focuses on three primary components:

1. **Vulnerability Detection**

- Compare CodeGuardian's LLM+static analysis pipeline with baseline tools (Bandit, Semgrep)
- Measure true positives, false positives, false negatives, and detection accuracy

2. Policy Enforcement

- Verify that CodeGuardian correctly identifies violated security rules
- Confirm policy mappings with human reviewers

3. Remediation Quality

- Reviewers rate clarity, helpfulness, and correctness of remediation suggestions
- Ratings use a 1–5 Likert scale
- Include human-written fixes for blind comparison

6.3 Analysis Methods

We will apply the following techniques:

- Paired t-tests to compare CodeGuardian with static analysis baselines
- Wilcoxon signed-rank tests for subjective Likert ratings
- Inter-rater reliability analysis (Cohen's κ) to validate expert agreement
- Qualitative coding for summarizing written feedback

6.4 Expected Results

We expect CodeGuardian to:

- Improve vulnerability detection accuracy compared to static tools alone
- Provide clearer and more actionable remediation suggestions
- Apply security policies consistently across cases
- Receive positive usability feedback from developers

7 Conclusion

This project demonstrates that traditional static analysis tools alone are not sufficient for ensuring secure software development. CodeGuardian addresses these limitations by combining LLM-driven reasoning, policy-aware analysis, and real-time IDE integration to deliver clearer and more actionable vulnerability insights.

By unifying static scanners with OWASP, CERT, and organization-specific rules, CodeGuardian provides developers with consistent guidance that strengthens code security while reducing false positives. The proposed evaluation plan will help measure improvements in detection accuracy, policy compliance, and remediation quality.

Future work may expand language support, integrate additional policy frameworks, and enhance collaboration features for development teams, allowing CodeGuardian to evolve into a more comprehensive secure coding assistant.

References

[1] OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. OWASP, 2021. Available: https://owasp.org/www-project-top-ten/

[2] SEI CERT. *CERT Secure Coding Standards*. Carnegie Mellon University, 2023. Available: https://wiki.sei.cmu.edu/confluence/display/seccode

[3] OpenAI. *GPT-4 Technical Report*. 2023. Available: https://openai.com/research/gpt-4

[4] PyCQA. *Bandit: Python Security Linter*. Available: https://bandit.readthedocs.io/

[5] Semgrep Security. *Semgrep: Lightweight Static Analysis for Code Security*. Available: https://semgrep.dev/

[6] National Institute of Standards and Technology (NIST). *Secure Software Development Framework (SSDF)*. NIST Special Publication 800-218, 2022.

[7] OWASP Foundation. *OWASP SAMM – Software Assurance Maturity Model*. OWASP, 2020.

[8] S. Ramírez. *FastAPI Documentation*. Available: https://fastapi.tiangolo.com/

[9] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available: https://www.postgresql.org/

[10] Meta Platforms Inc. *React: A JavaScript Library for Building User Interfaces*. Available: https://react.dev/

[11] Microsoft Security. "Secure Coding with AI Assistance: Challenges and Best Practices." Microsoft Security Blog, 2024.

[12] Doshi-Velez, F., Kim, B. "Towards a Rigorous Science of Interpretable Machine Learning." *arXiv:1702.08608*, 2022.